Université de Rouen Normandie

Master 1 – Génie Informatique et Logiciel

Projet Gestion de Flotte

Rapport de Semaine 4

Services Conducteurs & Maintenance, Kafka, Gateway GraphQL

Étudiants :
Ahcene Amouchas
Florian Pepin

Encadrants :
Lydia & Luc

Année universitaire :
2025 – 2026

Semaine :
4 / 7

4 avril 2026

Table des matières

1 Introduction **2**

2 Nouveaux services métier **3**
2.1 Service Driver 3
2.2 Service Maintenance 3
2.3 Données de démo (Datafaker) 3

3 Communication événementielle entre services **5**

4 Gateway GraphQL et authentification **6**
4.1 Authentification Keycloak 6

5 Qualité : tests et couverture **7**
5.1 Collection Bruno 7
5.2 Couverture JaCoCo 7

1. Introduction

Après la mise en place de l'infrastructure (S2) et du microservice Véhicules (S3), la semaine 4 complète la plateforme métier. Deux nouveaux services ont été développés — **Conducteurs** et **Maintenance** — et connectés aux autres via Kafka. Une passerelle GraphQL (Apollo Server) unifie l'accès aux trois services, protégée par Keycloak. Chaque service produit des données de démo au démarrage grâce à Datafaker, et la collection de tests Bruno couvre l'ensemble des endpoints.

2. Nouveaux services métier

Les services **Driver** et **Maintenance** suivent la même structure que le service Véhicules : API REST Spring Boot, base PostgreSQL dédiée, DDL géré par Hibernate (`ddl-auto: update`), et publication d'événements Kafka. Une instance PostgreSQL unique héberge les trois bases (`vehicle_db`, `driver_db`, `maintenance_db`) avec isolation logique au niveau base de données.

2.1. Service Driver

Le service gère le cycle de vie des conducteurs : profils, permis, et coordination avec le service Véhicules lors des assignations. Le modèle repose sur une relation **1-N** entre `Driver` et `DriverLicense`, avec suppression logique via `@SQLDelete` / `@SQLRestriction` Hibernate.

```java
@Entity @Table(name = "drivers")
@SQLDelete(sql = "UPDATE drivers SET deleted_at = NOW() WHERE id = ?"
    )
@SQLRestriction("deleted_at IS NULL")
public class Driver {
    @Id @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;
    private String firstName, lastName, email, phone, employeeId;

    @Enumerated(EnumType.STRING)
    private DriverStatus status = DriverStatus.ACTIVE;

    @OneToMany(mappedBy = "driver", cascade = CascadeType.ALL)
    private List<DriverLicense> licenses = new ArrayList<>();
}
```

Listing 1 – Entité Driver (extraits)

Une tâche `@Scheduled` (cron quotidien) détecte les permis expirant sous 30 jours et publie des événements Kafka (`LICENSE_EXPIRING`, `LICENSE_EXPIRED`) consommables par les autres services.

2.2. Service Maintenance

Le service assure la traçabilité complète des interventions, de la planification à la clôture. L'entité `MaintenanceRecord` utilise trois énumérations (`MaintenanceType`, `MaintenanceStatus`, `MaintenancePriority`). Le lien vers le véhicule est un simple UUID — pas de clé étrangère inter-bases, conformément au principe microservices. Une tâche `@Scheduled` quotidienne (01h00) passe les maintenances en retard au statut `OVERDUE` et publie une alerte Kafka.

2.3. Données de démo (Datafaker)

Plutôt que des fichiers `data.sql` statiques, chaque service embarque un `DataSeeder` activé par le profil Spring `dev`. Le `vehicle-service` insère d'abord un véhicule démo à UUID fixe via un `INSERT` SQL natif (nécessaire car un UUID assigné manuellement

ne passe pas par le cycle JPA `@GeneratedValue`), puis complète avec neuf véhicules Faker si la base n'est pas déjà peuplée.

Service	**Données générées**	**Détails**
vehicle-service	10 véhicules (1 démo + 9 aléatoires)	UUID fixe garanti ; Faker ignoré si base déjà
driver-service	8 conducteurs	Noms, emails, téléphones + 1 permis chacun
maintenance-service	12 maintenances	Types, priorités, dates, coûts pour les complé

Table 1 – Données injectées par Datafaker (profil `dev`)

3. Communication événementielle entre services

> **Principe :** Saga par **chorégraphie** — chaque service réagit aux événements des autres et publie ses propres résultats. Pas d'orchestrateur central.

Les événements circulent dans une enveloppe commune `KafkaEventEnvelope` qui embarque l'identifiant, le type, l'horodatage, le payload métier et les métadonnées de la source :

```
{
  "eventId": "625c87b9-...",
  "eventType": "MAINTENANCE_COMPLETED",
  "timestamp": 1775220800.293,
  "payload": { "vehicle_id": "39e75ab3-...", "cost_eur": 250.50 },
  "metadata": { "triggered_by": "maintenance-service" }
}
```

Listing 2 – Enveloppe événement unifiée

Le flux principal illustre la coordination entre les services via une saga compensatoire : Maintenance publie `MAINTENANCE_STARTED` (T1), Vehicle consomme et passe en `IN_MAINTENANCE` (T2). En cas d'échec, Vehicle publie `MAINTENANCE_REJECTED` qui déclenche l'annulation de la fiche côté Maintenance (C1).

Topic	Événements	Effet
`flotte.maintenance.events`	`STARTED`, `COMPLETED`, `CANCELLED`	Vehicle : `available`
`flotte.conducteurs.events`	`DRIVER_CREATED/UPDATED`, `LICENSE_EXPIRING/EXPIRED`	Notification int
`flotte.assignments.events`	`VEHICLE_ASSIGNED/UNASSIGNED`	Driver : `ON_TOU`

Table 2 – Topics Kafka et flux d'événements

4. Gateway GraphQL et authentification

> **Objectif :** Exposer une API GraphQL unifiée (Apollo Server, Node.js, port 4000) devant les trois services métier, protégée par des JWT Keycloak.

Le Gateway route les requêtes via des clients REST Axios vers chaque service backend et **forwarde** le header `Authorization` reçu sans le modifier. Un intercepteur Axios assure la résilience au démarrage (`ECONNREFUSED` / `ECONNRESET`) avec retry automatique.

Les relations entre domaines sont résolubles en une seule requête GraphQL : historique de maintenance depuis une fiche véhicule, profil du conducteur assigné, technicien d'une intervention. Cela évite au client d'interroger plusieurs services séparément.

4.1. Authentification Keycloak

Keycloak (v26.5.7) est déployé avec `-import-realm` pour charger automatiquement le realm `gestion-flotte` au démarrage. Les services Spring Boot valident les JWT via `jwk-set-uri` ; le Gateway se contente de transmettre le token reçu.

```
POST /auth/realms/gestion-flotte/protocol/openid-connect/token
  grant_type=password  client_id=admin-cli
  username=test-user   password=password
```

Listing 3 – Récupération du token (grant type password)

La base de Keycloak diffère selon le profil de déploiement : sous Kubernetes elle utilise `keycloak_db` (créée par le script `initdb` Helm) ; sous Docker Compose elle se connecte à la base `postgres` par défaut.

5. Qualité : tests et couverture

5.1. Collection Bruno

La collection de tests remplace Postman par des fichiers `.bru` versionnés dans Git. Un script `pre-request` dans `collection.bru` obtient et renouvelle le JWT automatiquement avant chaque requête via `bru.sendRequest`, en lisant la date d'expiration dans le payload du token — aucune requête d'authentification manuelle n'est nécessaire. Deux environnements sont fournis : **Docker** (`http://127.0.0.1:4000`, Keycloak port 8180) et **Minikube** (`http://flotte.local`, Ingress K8s).

Service	Endpoints testés	Résultat
Driver	8 endpoints REST + Query GraphQL `drivers`	18/18 OK
Maintenance	5 endpoints REST + Query GraphQL `maintenanceRecords`	18/18 OK
Vehicle	5 endpoints REST + Query GraphQL `vehicles`	18/18 OK

Table 3 – Bilan Bruno — 18 endpoints REST + 3 queries GraphQL, tous verts

5.2. Couverture JaCoCo

Les tests unitaires (JUnit 5, Mockito, `@WebMvcTest`) et les tests d'intégration (`@SpringBootTest`, H2) sont séparés via le profil Maven `unit-coverage`, ce qui permet d'isoler le rapport JaCoCo sur les tests unitaires.

driver-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.flotte.driver.config		0%		0%	12	12	22	22	10	10	1	1
com.flotte.driver.controllers		55%		n/a	4	9	4	12	4	9	0	1
com.flotte.driver.models		88%		n/a	4	40	6	54	4	40	0	2
com.flotte.driver		37%		n/a	1	2	2	3	1	2	0	1
com.flotte.driver.events		98%		n/a	1	13	1	23	1	13	0	7
com.flotte.driver.services		100%		75%	4	23	0	103	0	15	0	1
com.flotte.driver.events.producers		100%		n/a	0	5	0	37	0	5	0	1
com.flotte.driver.dto		100%		n/a	0	6	0	6	0	6	0	6
com.flotte.driver.events.consumers		100%		100%	0	6	0	22	0	4	0	2
com.flotte.driver.models.enums		100%		n/a	0	2	0	13	0	2	0	2
com.flotte.driver.jobs		100%		n/a	0	5	0	17	0	5	0	1
Total	151 of 1,392	89%	8 of 24	66%	26	123	35	312	20	111	1	25

Figure 1 – JaCoCo — `driver-service`

maintenance-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.flotte.maintenance.config		0%		0%	11	11	19	19	9	9	1	1
com.flotte.maintenance.service		91%		62%	23	47	4	83	1	18	0	1
com.flotte.maintenance.events		93%		60%	7	30	3	56	3	25	1	7
com.flotte.maintenance.model		90%		n/a	5	36	5	54	5	36	0	4
com.flotte.maintenance.controller		85%		n/a	1	7	1	9	1	7	0	1
com.flotte.maintenance		37%		n/a	1	2	2	3	1	2	0	1
com.flotte.vehicle.events		100%		n/a	0	4	0	4	0	4	0	4
com.flotte.maintenance.dto		100%		n/a	0	3	0	3	0	3	0	3
Total	177 of 1,215	85%	30 of 72	58%	48	140	34	231	20	104	2	22

Figure 2 – JaCoCo — `maintenance-service`

vehicle-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.flotte.vehicle.config		0%		0%	12	12	23	23	10	10	1	1
com.flotte.vehicle.services		88%		58%	12	26	2	127	7	20	0	1
com.flotte.vehicle.models		88%		n/a	5	45	5	52	5	45	0	2
com.flotte.vehicle.events.consumers		88%		58%	4	13	3	35	0	5	0	1
com.flotte.vehicle		37%		n/a	1	2	2	3	1	2	0	1
com.flotte.vehicle.events		98%		n/a	1	14	1	23	1	14	0	6
com.flotte.vehicle.dto		100%		n/a	0	7	0	15	0	7	0	6
com.flotte.vehicle.events.producers		100%		n/a	0	5	0	16	0	5	0	1
com.flotte.vehicle.controllers		100%		n/a	0	10	0	13	0	10	0	1
com.flotte.vehicle.models.enums		100%		n/a	0	2	0	10	0	2	0	2
com.flotte.maintenance.model		100%		n/a	0	1	0	6	0	1	0	1
com.flotte.maintenance.events		100%		n/a	0	1	0	1	0	1	0	1
Total	202 of 1,509	86%	14 of 28	50%	35	138	36	324	24	122	1	24

FIGURE 3 – JaCoCo — `vehicle-service`

Les trois services dépassent l'objectif de 80% de couverture en lignes : `driver-service` et `vehicle-service` atteignent environ 89%, `maintenance-service` environ 85%.